Cognitive Immunization: A Multi-Layer Defense-in-Depth Architecture for LLM Safety via Representation Engineering

Salar Farzaneh[1]

[1]Independent Researcher , ORCID: 0009-0008-7236-9241

Preprint v1 — July 2026

Abstract

Large Language Model (LLM) deployment in agentic settings remains vulnerable to jailbreaking, and current defenses are largely reactive, addressing surface-level failures without accounting for the underlying representational structure of transformer models. We propose the **Cognitive Immunization Framework (CIF)**, a four-layer defense-in-depth architecture combining input filtering, prompt neutralization, sparse-autoencoder-based representation steering, and output validation. We motivate the representation-steering layer with a simple, testable hypothesis — that harmful-generation likelihood correlates with the proximity of residual-stream activations to a learned "harmful" direction — and show that, under this hypothesis, orthogonal projection away from that direction is a natural mitigation. We do *not* claim this hypothesis is proven; we state it explicitly as an assumption motivating the architecture, distinct from the elementary linear-algebra fact that orthogonal projection zeroes a specific activation component. We describe a low-cost, reproducible evaluation protocol using 4-bit-quantized open-weight models on free-tier GPUs, and report the current status of that evaluation, which is incomplete at the time of writing. This paper is offered as a theoretical and architectural contribution with an accompanying reference implementation, not as a validated empirical result.

Keywords: LLM Safety, Jailbreak Defense, Sparse Autoencoders, Representation Engineering, Defense-in-Depth

1 Introduction

Jailbreaking — the elicitation of harmful or policy-violating outputs from aligned LLMs — remains an open problem as these models are given more autonomy and tool access. Existing defenses generally fall into three families: input/output filtering, safety fine-tuning (RLHF/DPO), and, more recently, representation engineering. Each has known weaknesses: filters are bypassed by semantic obfuscation, fine-tuned safety behavior can be eroded by further fine-tuning or multi-turn "soft" attacks, and single-point representation interventions have not been shown to generalize robustly on their own.

This paper's contribution is architectural and hypothesis-generating rather than a proof of a new law of LLM behavior. We propose combining four complementary, individually well-motivated defenses into one pipeline, argue informally why layering them raises the cost of adaptive attacks, and lay out — with an explicit hypothesis, not a postulate — why a representation-steering layer based on sparse autoencoders (SAEs) is a reasonable mechanism. We are transparent that:

(1) The core theoretical claim (Section 4) is a *hypothesis*, stated for the first time here, not a validated scientific law.

(2) The only rigorously *proven* statement in this paper is a linear-algebra fact about orthogonal projection (Section 4.2), which is necessary but not sufficient to establish the safety claim.

(3) Empirical validation is in progress and results are not yet available; Section 5 reports protocol and status, not findings.

Contributions

1. An explicit, falsifiable hypothesis connecting residual-stream geometry to harmful-generation probability, stated as an assumption to be tested — not asserted as fact.
2. A four-layer defense-in-depth architecture combining input classification, prompt rewriting, SAE-based activation steering, and output validation, with a described cross-layer feedback mechanism.
3. An informal cost argument for why combining differentiable and non-differentiable defense stages raises the difficulty of gradient-based adaptive attacks (not a formal complexity proof).
4. A low-cost, reproducible evaluation protocol runnable on free-tier hardware, intended to make the hypothesis testable by others regardless of compute budget.

2 Background and Related Work

The adversarial landscape spans direct prompt injection, persona and multi-turn cognitive chains, adversarial suffix attacks [5], and cross-lingual evasion; recent work shows even state-of-the-art models remain vulnerable shortly after release. Current defenses — input/output filtering [1], safety training [3], and static representation engineering [4] — operate largely as isolated silos and are vulnerable to non-linear adversarial drift in deeper transformer layers. Fine-tuning has separately been shown to erode safety alignment even when not adversarially intended [2].

CIF's distinguishing claim relative to this literature is architectural: it layers mechanisms with different failure modes (differentiable vs. non-differentiable, input-side vs. activation-side vs. output-side) rather than proposing a single new mechanism.

3 The Cognitive Immunization Framework (CIF)

CIF is a middleware shield consisting of four sequential, complementary layers.

3.1 Threat Model

We consider an adaptive adversary with varying degrees of access: a **black-box** attacker who queries the API using transferable suffix attacks or multi-turn manipulation, and a **white-box** attacker with full knowledge of the CIF pipeline (weights, SAE architecture) who can compute gradients through the stack. The attacker may use gradient-based optimization (e.g., GCG, AutoDAN) and semantic obfuscation, but cannot modify the deployed target LLM or SAE probes at runtime.

3.2 Layer 1: Semantic-Syntactic Intent Bounding

A fine-tuned classifier categorizes prompts, and Semantic Anomaly Detection computes the Mahalanobis distance D_M of the input embedding x from the benign distribution μ:

$$D_M(x) = \sqrt{(x-\mu)^T \Sigma^{-1} (x-\mu)}$$

Trajectory Intent Graphing maintains a directed graph $\mathcal{G}_t = (V_t, E_t)$ for multi-turn conversations, tracking cumulative drift Δ_t of intent embeddings; if drift toward a banned direction exceeds a threshold θ_{drift}, the conversation is terminated.

3.3 Layer 2: Constitutional Prompt Neutralization

This layer rewrites the user prompt into a safer version using a smaller aligned LLM guided by N principles, seeking

$$p_{safe} = \arg\min_{p'} \Big[\alpha \cdot \text{BERTScore}(p', p_{orig}) + (1-\alpha) \cdot \cos(E(p'), \mathcal{H}) \Big],$$

where $\mathcal{H}$ denotes the region of embedding space associated with harmful content. Minimizing cosine similarity to $\mathcal{H}$ is intended to move the target LLM's initial activations away from that region.

3.4 Layer 3: Runtime Representation Surgery via SAEs

A Top-K Sparse Autoencoder (SAE) trained on the residual stream decomposes activation h_l at layer l into sparse features z:

$$\mathcal{L}_{SAE} = \|h_l - W_{dec}\,\sigma(W_{enc}h_l + b)\|_2^2 + \lambda\|z\|_1$$

The harmful direction $f_{harm} \in \mathbb{R}^d$ is estimated as the difference in means between harmful-compliant and benign-refusal activation sets:

$$f_{harm} = \mathbb{E}[z_{harm}] - \mathbb{E}[z_{benign}]$$

During generation, if a linear probe $P(z) > \tau_{act}$, the activation is projected orthogonally away from f_{harm}:

$$h_l' = h_l - \frac{\langle h_l, f_{harm}\rangle}{\|f_{harm}\|^2}\, f_{harm}$$

Note on the reference implementation. The architecture above specifies f_{harm} as derived from trained SAE features z. The reference implementation accompanying this preprint (Appendix A) instead computes f_{harm} directly from raw residual-stream activations via Difference-in-Means, without first training an SAE — a lighter-weight approximation consistent with established direction-finding techniques in the representation-engineering literature. This is a deliberate simplification for the free-tier evaluation protocol in Section 5, not a silent substitution: it removes the sparse decomposition step ($\mathcal{L}_{SAE}$ above) entirely, so the probe threshold τ_{act} and the projection are applied to raw activations rather than SAE features. Whether full SAE-based decomposition adds robustness beyond the raw-activation direction is an open, empirically testable question we flag here rather than assume; a controlled comparison of both variants is planned as part of the evaluation in Section 5.

3.5 Layer 4: Output Validation and Backtracking

Generated text is inspected by a safety classifier (e.g., Llama-Guard-3). If a violation is detected, the system backtracks and regenerates with stricter safety instructions, or falls back to a generic refusal.

4 Theoretical Motivation

4.1 The Latent Bounding Hypothesis

We introduce, as a hypothesis rather than a postulate or law, the following assumption motivating Layer 3:

> **Latent Bounding Hypothesis (informal).** For a given LLM, the probability of harmful generation at a token step correlates with the proximity of the residual-stream activation to a "harmful manifold" — a region of activation space associated with harmful completions.

We state explicitly that this hypothesis is *not derived* from first principles; it is an assumption consistent with the broader representation-engineering literature, stated here in a form specific to this architecture. It has *not* been empirically validated in this paper. Section 5 proposes a direct test (the Latent Distance Metric), but results are pending. Until tested, all downstream claims that depend on it should be read as *conditional* on the hypothesis holding, not as established fact.

4.2 A Proven Fact: Orthogonal Projection Zeroes the Target Component

The following is a straightforward linear-algebra fact, proven for completeness, and should not be conflated with a safety guarantee.

Claim 1. *Given activation h_t and direction f_{harm}, the projection*

$$h'_t = h_t - \frac{\langle h_t, f_{harm} \rangle}{\|f_{harm}\|^2} f_{harm}$$

satisfies $\langle h'_t, f_{harm} \rangle = 0$.

Proof. By direct substitution,

$$\langle h'_t, f_{harm} \rangle = \langle h_t, f_{harm} \rangle - \frac{\langle h_t, f_{harm} \rangle}{\|f_{harm}\|^2} \langle f_{harm}, f_{harm} \rangle = \langle h_t, f_{harm} \rangle - \langle h_t, f_{harm} \rangle = 0. \qquad \square$$

What this does and does not establish. This proves only that the component of h_t along f_{harm} is removed. It does *not* by itself prove anything about the distance to a manifold $\mathcal{H}$ that is not exactly the line spanned by f_{harm}, nor about downstream generation probabilities. The connection between this projection and actual harmful-output rates is precisely what Section 5's evaluation is designed to test, and is not established by this proof alone.

4.3 Informal Cost Argument

We retain the qualitative argument that combining differentiable stages (Layer 1 classifier, Layer 3 probe) with non-differentiable stages (Layer 2 rewriting, Layer 4 classification) likely prevents naive end-to-end gradient attacks (e.g., GCG) from optimizing through the full pipeline. We deliberately avoid asserting a specific combinatorial complexity bound, since no derivation was given connecting the four constraints to such a bound; this is stated as a qualitative, testable claim to be assessed via the red-teaming protocol in Section 5.3, not a proven result.

5 Evaluation Protocol and Current Status

5.1 Protocol

We target Llama-3-8B-Instruct and Mistral-7B-Instruct-v0.2, loaded with 4-bit NormalFloat (NF4) quantization via `bitsandbytes` to fit within a 16GB free-tier GPU (e.g., a single Colab T4). SAE probes are trained on unquantized residual-stream activations; inference is performed on the 4-bit representation.

5.2 Status

As of this writing, the following remain incomplete and are reported honestly as such:

- SAE probe training — *not yet complete*
- Baseline Attack Success Rate (ASR) on AdvBench/HarmBench (no defense) — *not yet run*
- ASR with CIF Layers 1–4 active — *not yet run*
- Latent Distance Metric (pre/post projection) — *not yet measured*
- Benign-capability degradation check — *not yet run*

No numeric results are reported in this version. A future version of this paper will report all of the above, including negative or weak results, before any claim of validation is made.

5.3 Proposed Metrics

Standard ASR on AdvBench (520 prompts) and HarmBench; the Latent Distance Metric $d(h_t, \mathcal{H})$ measured before and after Layer 3 projection, as a direct empirical test of the hypothesis in Section 4.1; and multi-turn agentic red-teaming over 10+ conversational turns.

6 Discussion

CIF is proposed as a candidate architecture, not a solved paradigm. Whether it meaningfully outperforms simpler baselines (e.g., a single SAE-steering layer alone, or an output classifier alone) is untested and should be a required baseline comparison in the full evaluation. A central risk to the framework's validity is that Layers 1, 2, and 4 might account for most of any observed robustness gain, with Layer 3 contributing little — this requires an ablation study, not just an end-to-end number.

6.1 Latency Overhead

Layers 2 and 4 each invoke a separate generative model (a smaller aligned LLM for prompt rewriting, and Llama-Guard-3 for output classification) in addition to the target model itself. Called naively and sequentially, this adds two full forward-generation passes to every request, which can substantially increase time-to-first-token and is a real deployment cost that the free-tier evaluation protocol in Section 5 does not yet measure. Mitigations we intend to evaluate rather than assume effective include: (i) gating Layer 2 rewriting behind the Layer 1 risk score so it only runs on prompts already flagged as ambiguous, skipping it for clearly benign traffic; (ii) replacing Layer 2's rewriting LLM and Layer 4's classifier with distilled or smaller models sized for low latency; (iii) running Layer 4 classification on a truncated response prefix with early-exit backtracking rather than waiting for full generation; and (iv) batching or overlapping Layer 2/4 calls with target-model decoding where the serving stack permits it. We report these as design options to be measured, not as a solved overhead problem.

7 Limitations

- No experimental results are yet available; all architectural claims are unvalidated.
- The Latent Bounding Hypothesis assumes harmful content can be isolated as a single linear direction in activation space. Real jailbreak behavior against complex, compositional, or multi-turn attacks may be multi-objective and non-linear, in which case a single direction f_{harm} would capture only part of the harmful subspace; this is a testable assumption, not a settled property of the model.
- The reference implementation's use of raw-activation Difference-in-Means in place of trained SAE features (noted in Section 3.3) means the empirical results this preprint will eventually report characterize that simpler variant, not the full SAE-based design; the two should not be treated as interchangeable until compared directly.
- The cost argument in Section 4.3 is qualitative; a white-box adversary aware of the full pipeline may find attack strategies not considered here.
- The latency overhead from sequential Layer 2/4 model calls (Section 6.1) is currently unmeasured and unmitigated in the reference implementation.
- No cross-layer ablation has yet been performed.

8 Conclusion

We present CIF as a candidate defense-in-depth architecture and an explicit, falsifiable hypothesis motivating its representation-steering layer, together with a low-cost evaluation protocol. We deliberately avoid claiming validation until the evaluation in Section 5 is complete and reviewed, and we invite replication and critique, including of the hypothesis itself.

References

[1] H. Inan et al., "Llama Guard: LLM-based Input-Output Safeguard," arXiv:2312.06674, 2023.

[2] X. Qi et al., "Fine-tuning Aligned LLMs Compromises Safety," arXiv:2310.03693, 2024.

[3] A. Robey et al., "SmoothLLM: Defending Large Language Models Against Jailbreaking Attacks," arXiv:2310.03684, 2023.

[4] Y. Zeng et al., "Defending Against Indirect Prompt Injection Attacks," arXiv:2405.00123, 2024.

[5] A. Zou et al., "Universal and Transferable Adversarial Attacks on Aligned LLMs," arXiv:2307.15043, 2023.

A Reference Implementation of Representation Surgery (Layer 3)

The following PyTorch forward hook implements the orthogonal projection described in Section 3.3. It is a reference implementation only; `load_pretrained_sae_probe()` is a placeholder pending completion of the SAE training described in Section 5.2.

```
import torch
import torch.nn as nn

class CIF_RepresentationSurgery(nn.Module):
   def __init__(self, f_harm_direction, tau_act=0.75):
      super().__init__()
      self.f_harm = f_harm_direction
      self.tau_act = tau_act
      self.sae_probe = load_pretrained_sae_probe() # placeholder

   def forward(self, module, input, output):
      h_t = output
      with torch.no_grad():
         z = self.sae_probe.encode(h_t)
         probe_confidence = self.sae_probe.classify(z)

      if probe_confidence.max() > self.tau_act:
         f = self.f_harm.to(h_t.device)
         dot = torch.sum(h_t * f, dim=-1, keepdim=True)
         norm_sq = torch.norm(f, p=2, dim=-1, keepdim=True) ** 2
         alpha = dot / (norm_sq + 1e-8)
         return h_t - alpha * f
      return h_t
```